Name: Khushi Chhaproo | Roll No: 07 | Div: D20C

Blockchain Experiment No. 5

Aim:

Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different
- sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.

Variables in Solidity can be state variables (stored on the blockchain permanently), local variables (temporary, created during function execution), or global variables (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

a. Function Visibility defines who can access a function:

- **public:** available both inside and outside the contract.
- **private:** only accessible within the same contract.
- **internal:** accessible within the contract and its child contracts.
- **external:** can be called only by external accounts or other contracts.

b. Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).

c. Constructors are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

4. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

if-else allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.

Loops (for, while, do-while) enable repeated potentially making the contract expensive to execute.execution of code. For example, iterating through an

5. Data Structures: Arrays, Mappings, Structs, Enums

6. Data Locations

Solidity uses three primary data locations for storing variables:

storage: Data stored permanently on the blockchain.

Examples: state variables.

memory: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs gas costs and performance efficiency in contract design very important Functions like transfer() and send() are commonly

Output:

The hands-on tutorials were carried out in the Remix IDE. Each tutorial step is shown below.

Tutorial No. 1

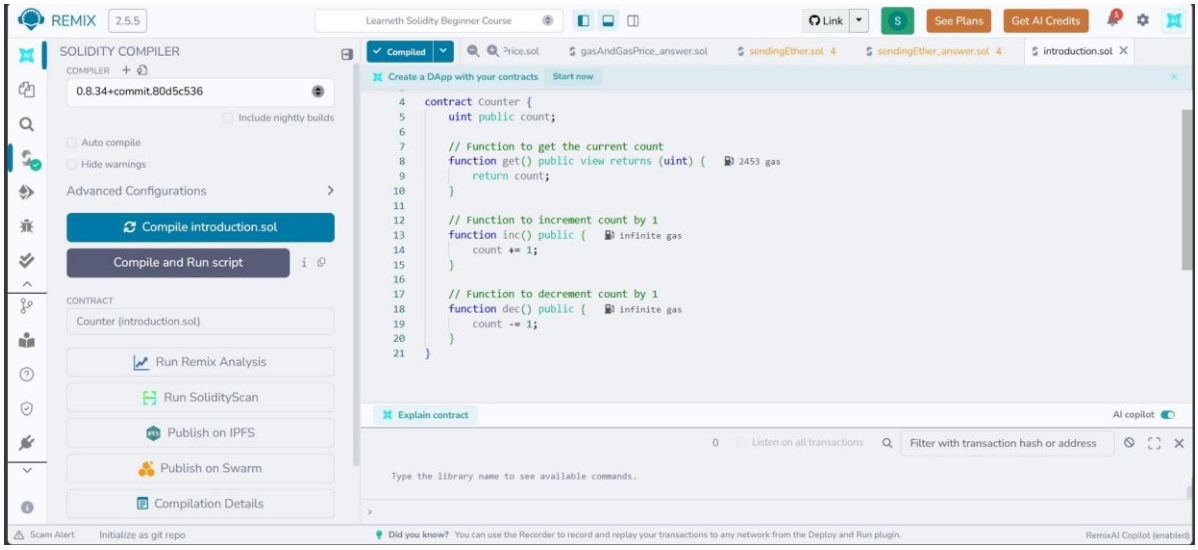


Figure 1 — Tutorial No. 1 execution in Remix IDE. **Tutorial**

No. 2

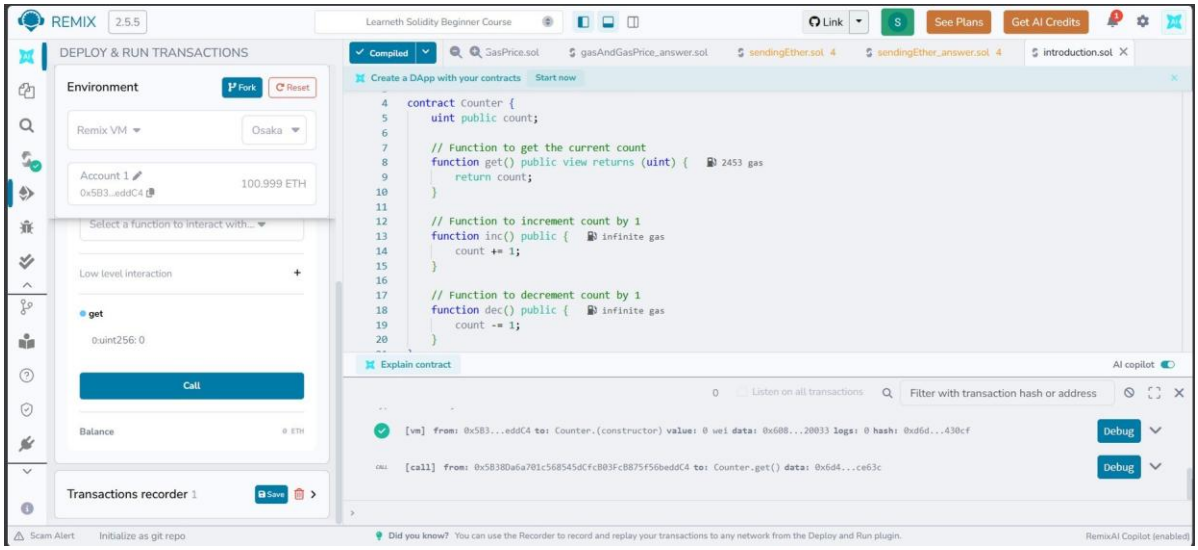


Figure 2 — Tutorial No. 2 execution in Remix IDE.

Tutorial No. 3

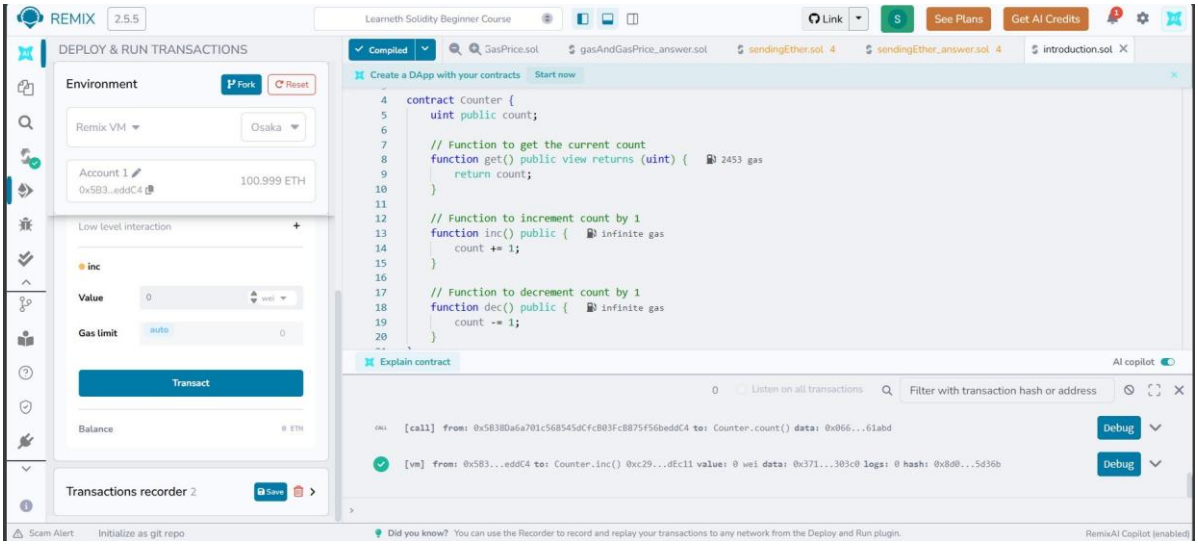


Figure 3 — Tutorial No. 3 execution in Remix IDE. **Tutorial**

No. 4

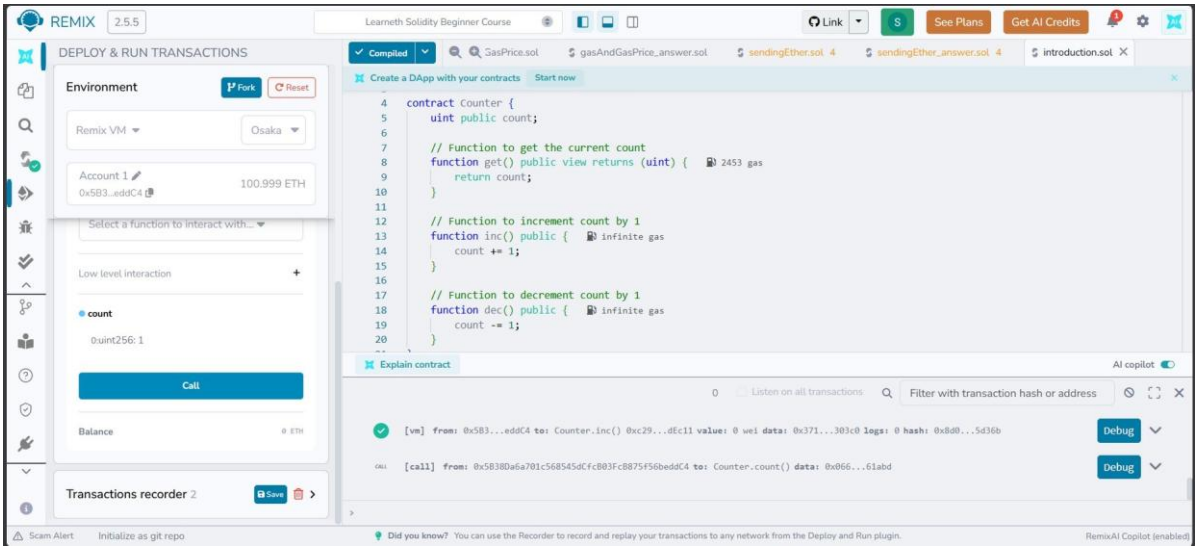


Figure 4 — Tutorial No. 4 execution in Remix IDE.

Tutorial No. 5

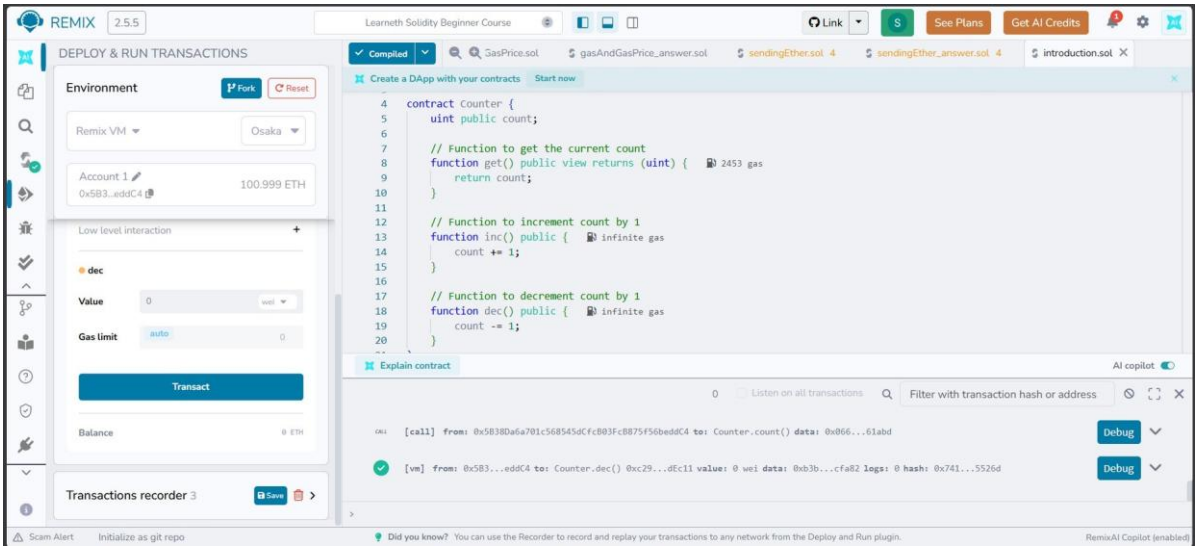


Figure 5 — Tutorial No. 5 execution in Remix IDE. **Tutorial**

No. 6

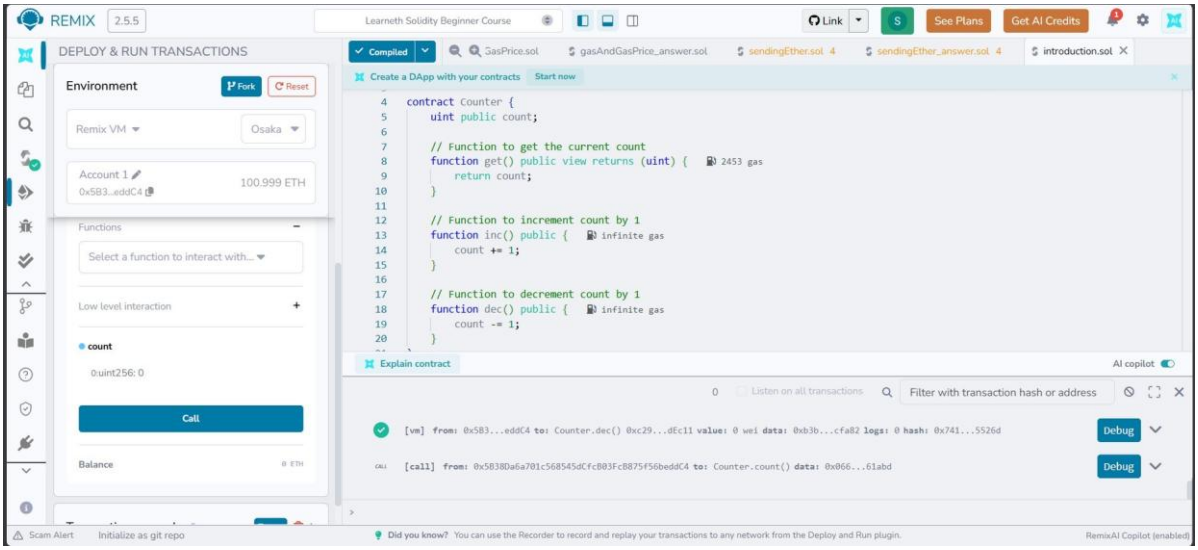


Figure 6 — Tutorial No. 6 execution in Remix IDE.

Tutorial No. 7

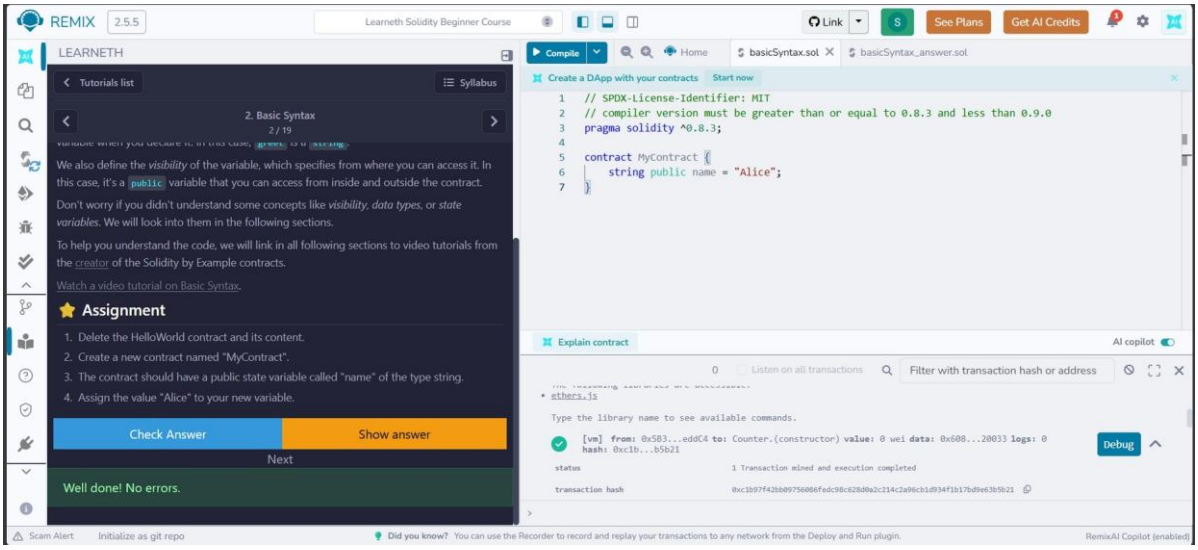


Figure 7 — Tutorial No. 7 execution in Remix IDE. **Tutorial**

No. 8

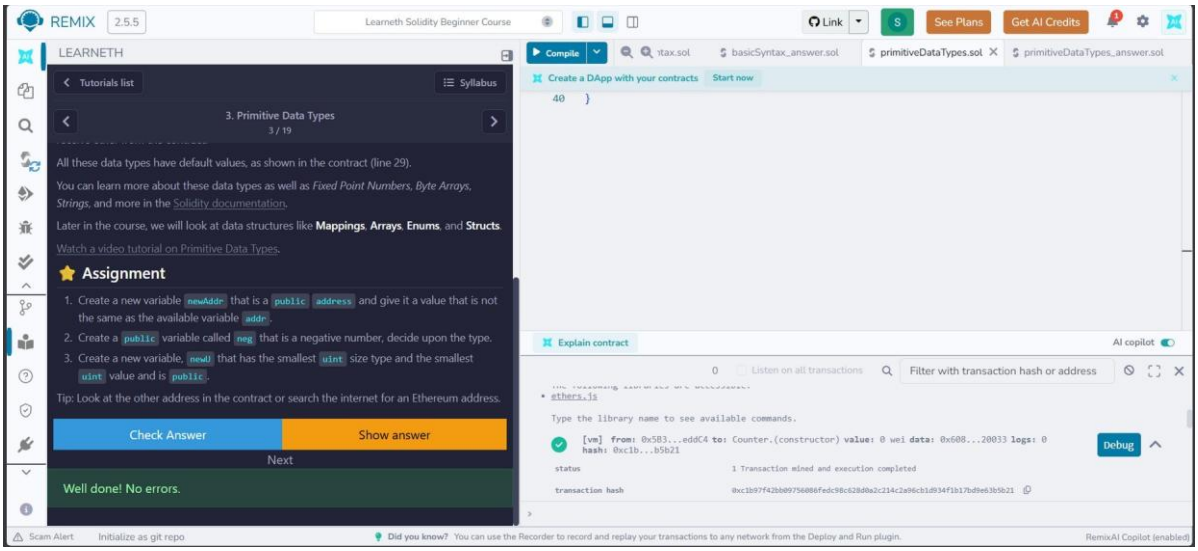


Figure 8 — Tutorial No. 8 execution in Remix IDE.

Tutorial No. 9

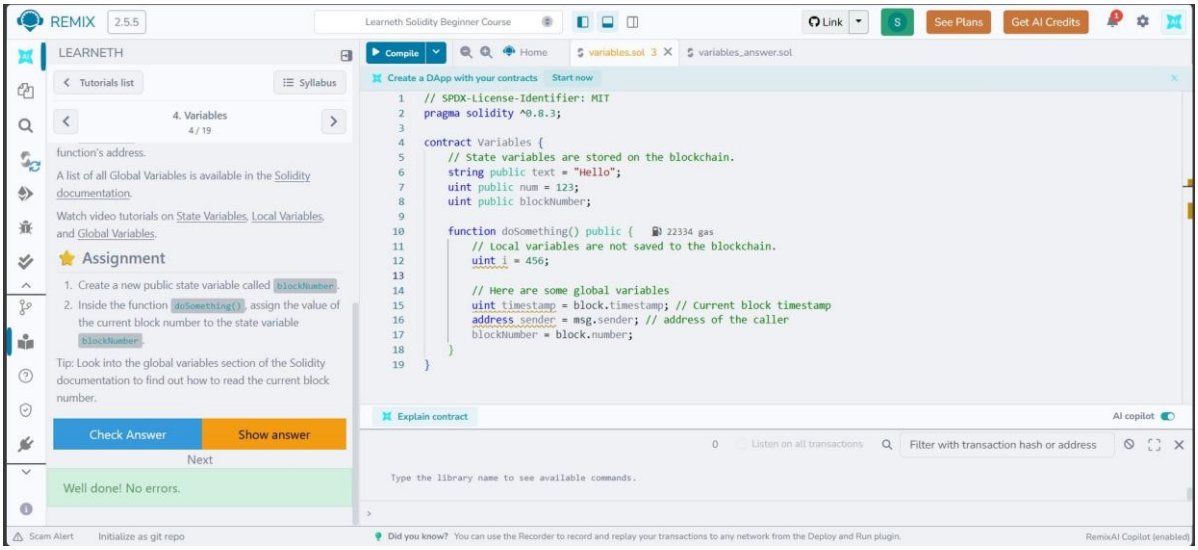


Figure 9 — Tutorial No. 9 execution in Remix IDE. **Tutorial**

No. 10

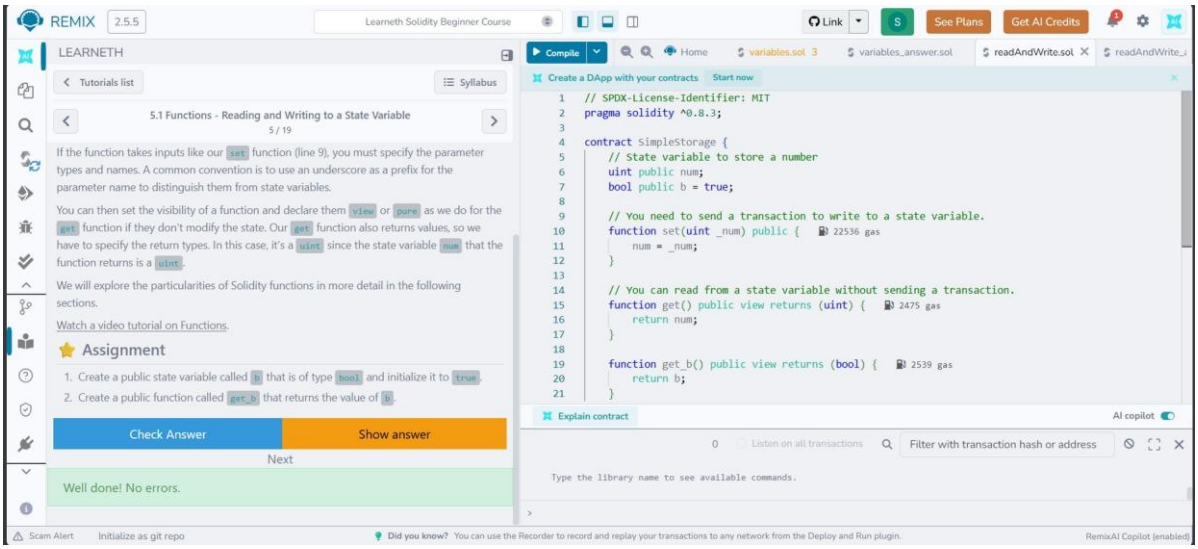


Figure 10 — Tutorial No. 10 execution in Remix IDE.

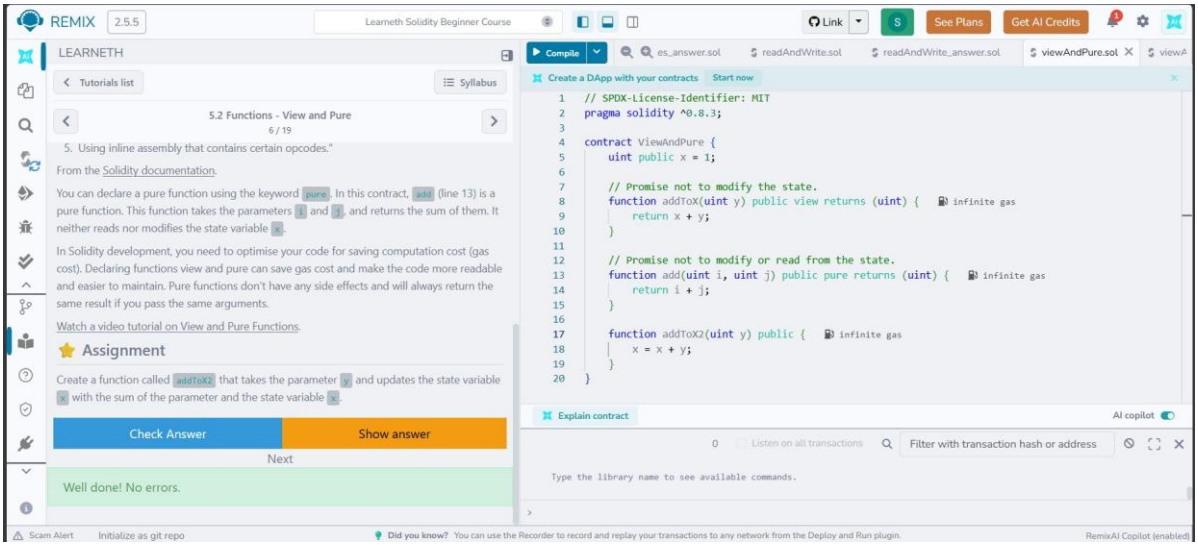


Figure 11 — Tutorial No. 10 execution in Remix IDE. **Tutorial**

No. 11

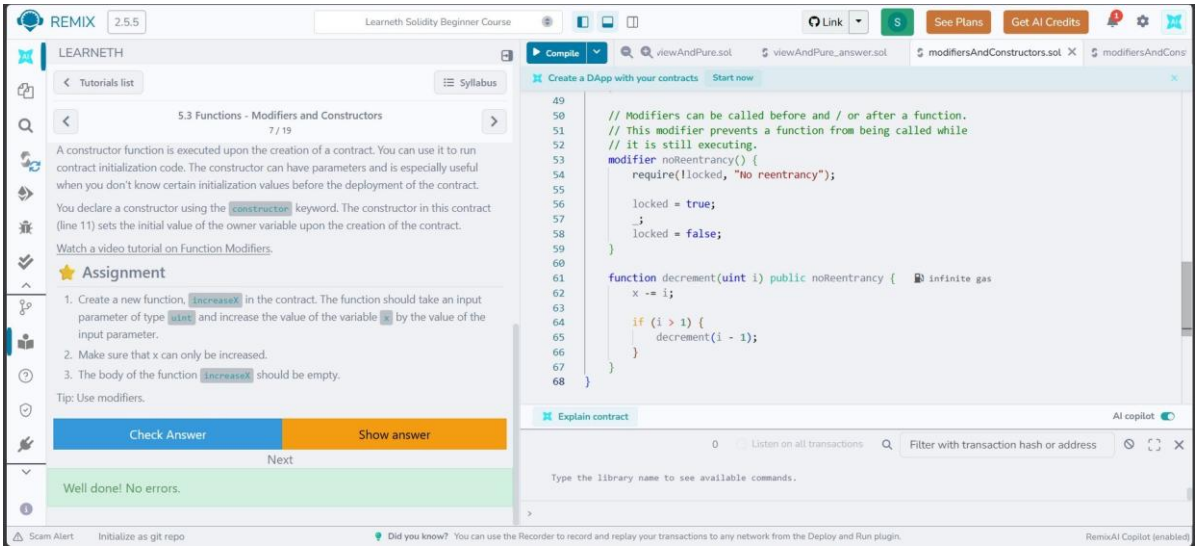


Figure 12 — Tutorial No. 11 execution in Remix IDE.

Tutorial No. 12

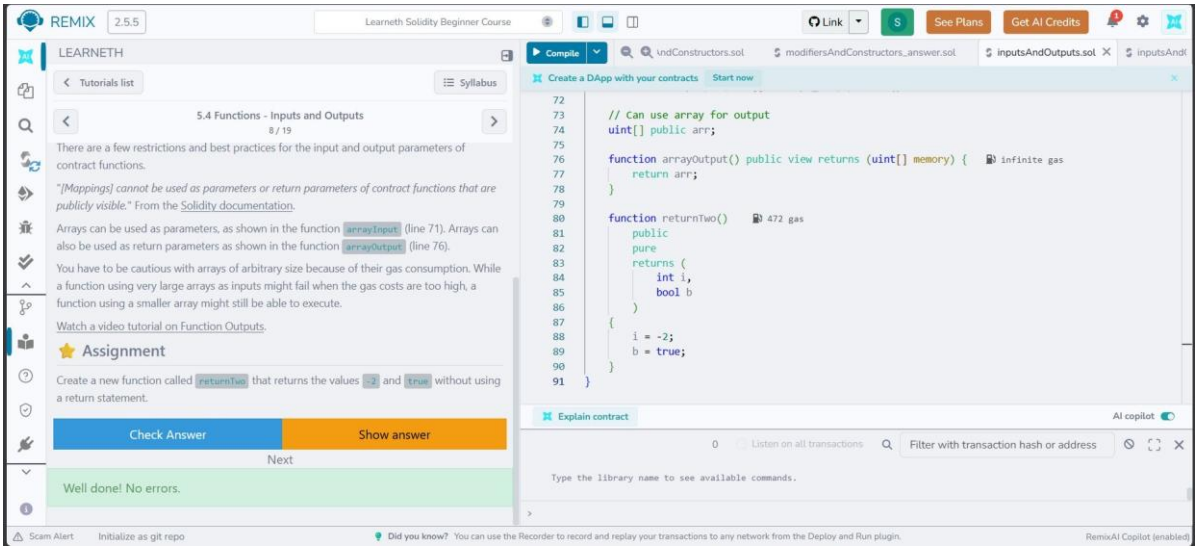


Figure 13 — Tutorial No. 12 execution in Remix IDE. **Tutorial**

No. 13

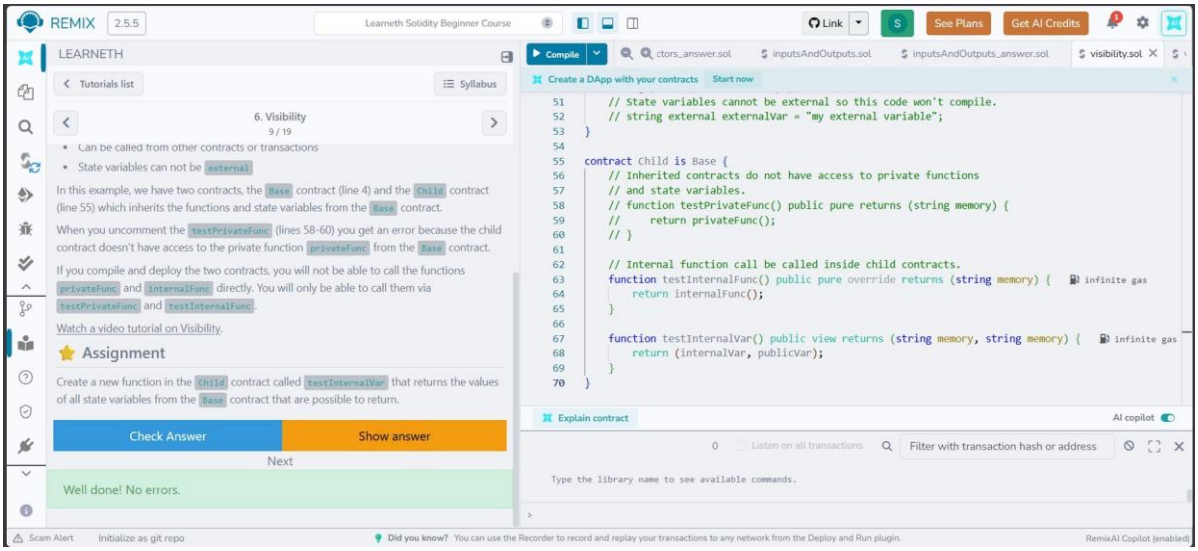


Figure 14 — Tutorial No. 13 execution in Remix IDE.

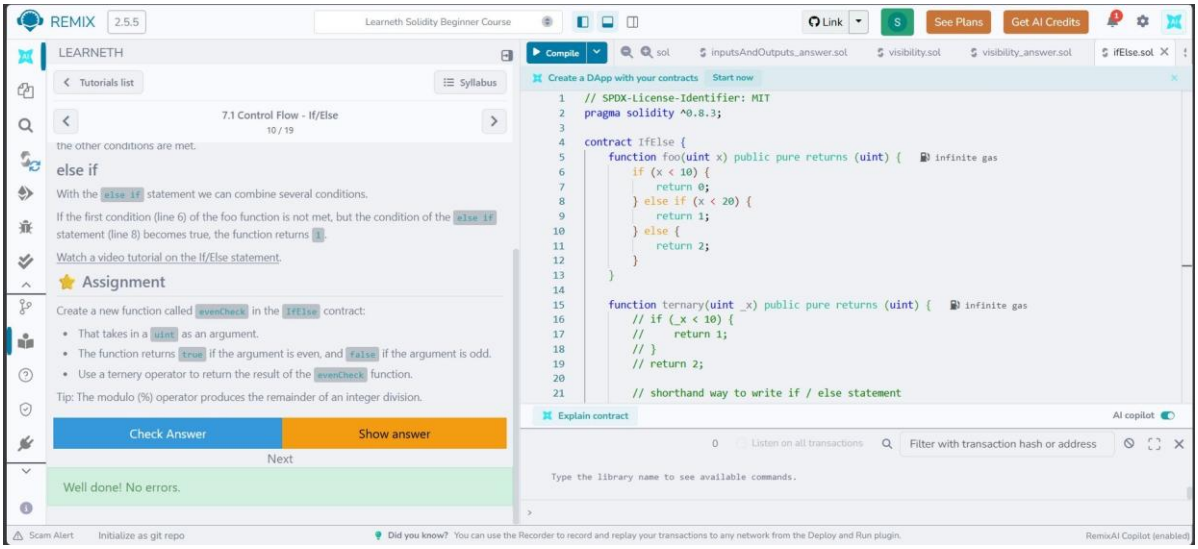


Figure 15 — Tutorial No. 13 execution in Remix IDE. **Tutorial**

No. 14

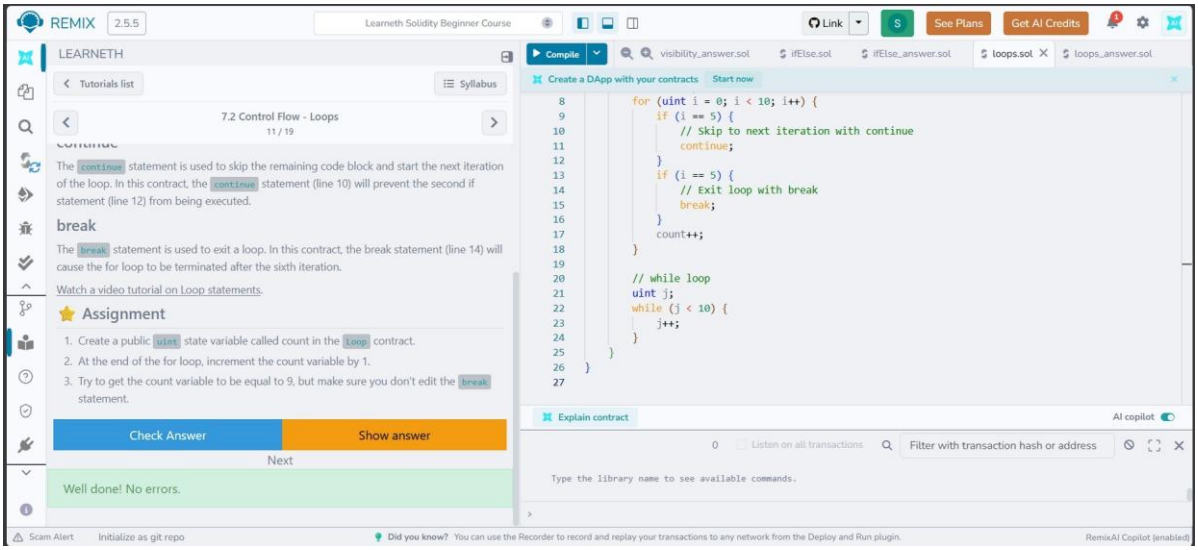


Figure 16 — Tutorial No. 14 execution in Remix IDE.

Tutorial No. 15

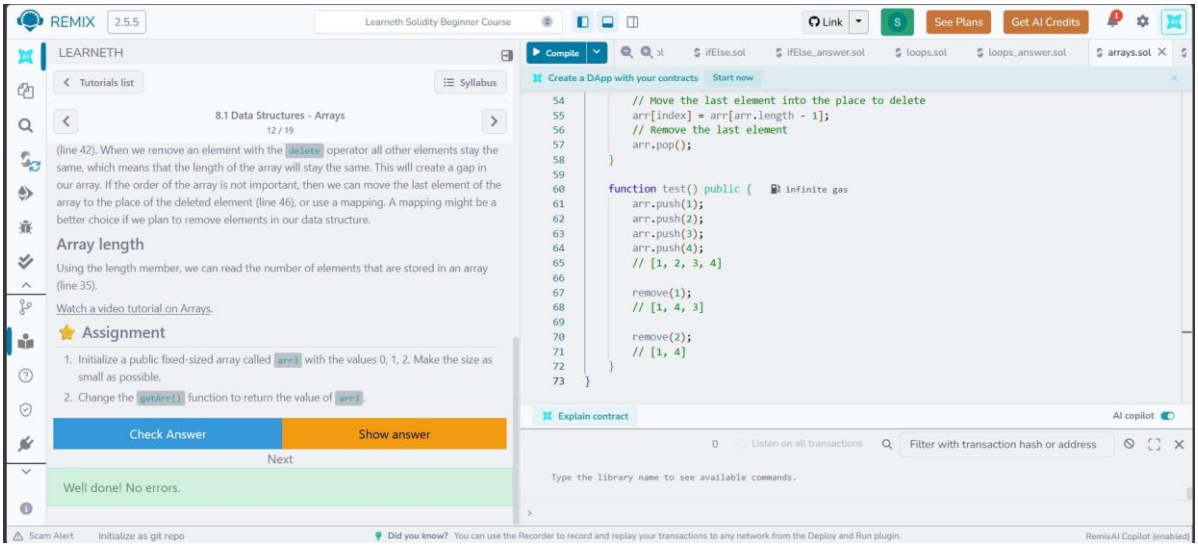


Figure 17 — Tutorial No. 15 execution in Remix IDE. **Tutorial**

No. 16

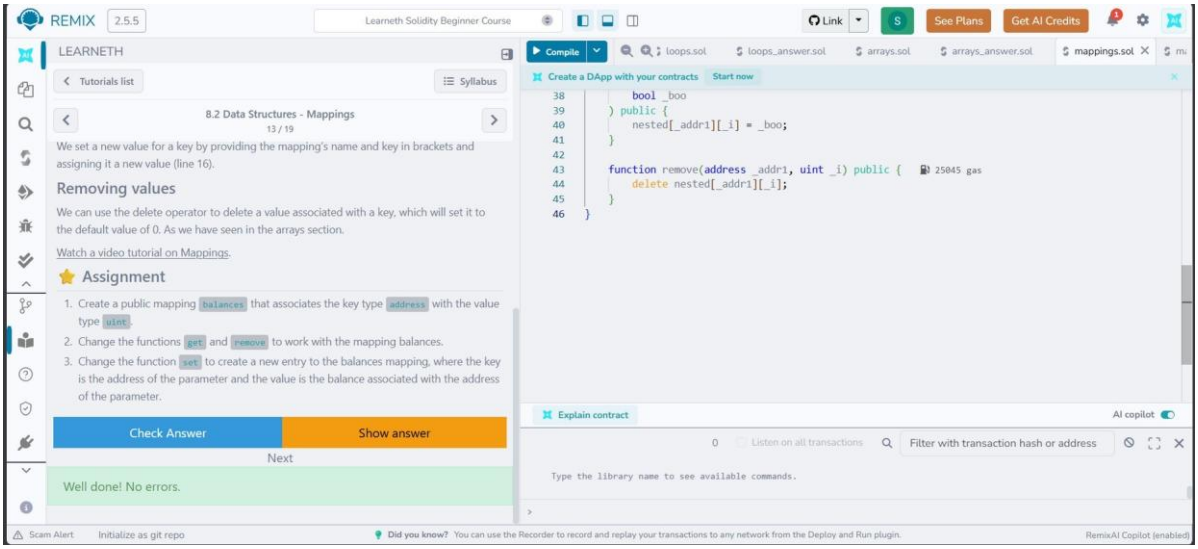


Figure 18 — Tutorial No. 16 execution in Remix IDE.

Tutorial No. 17

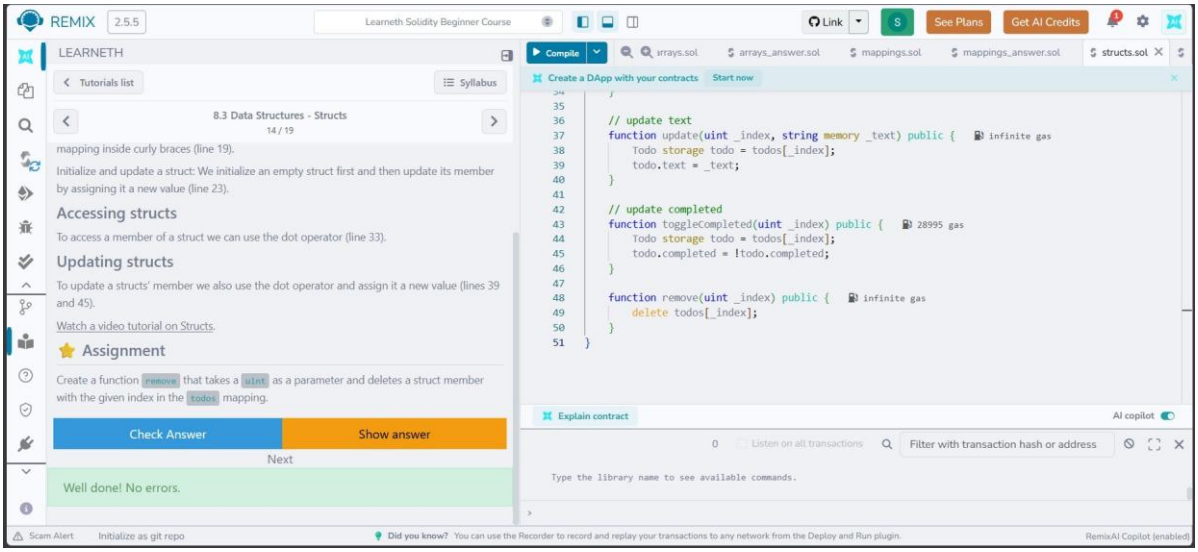


Figure 19 — Tutorial No. 17 execution in Remix IDE. **Tutorial**

No. 18

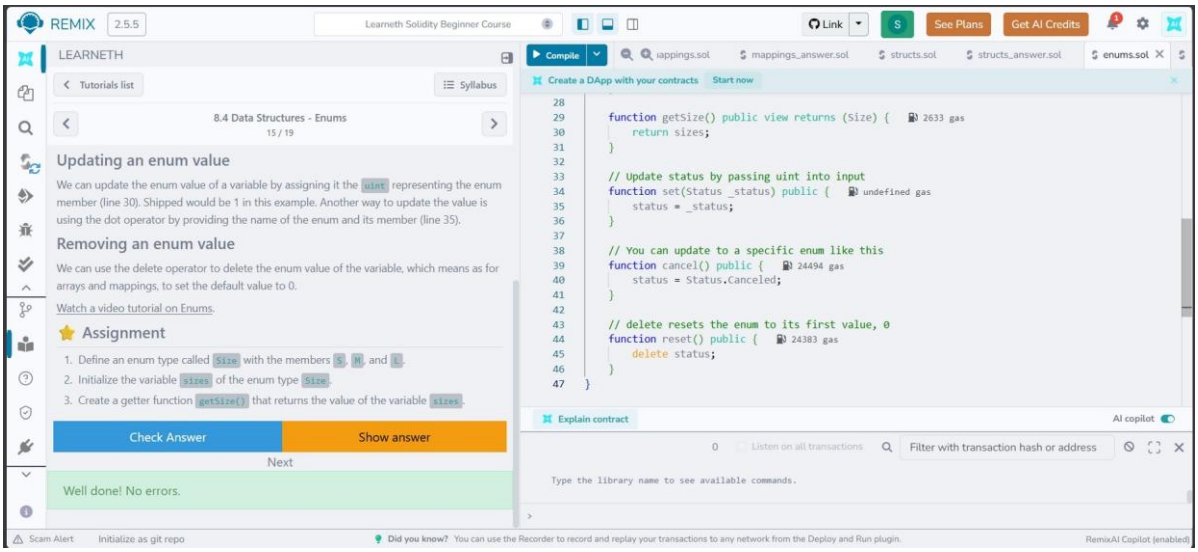


Figure 20 — Tutorial No. 18 execution in Remix IDE.

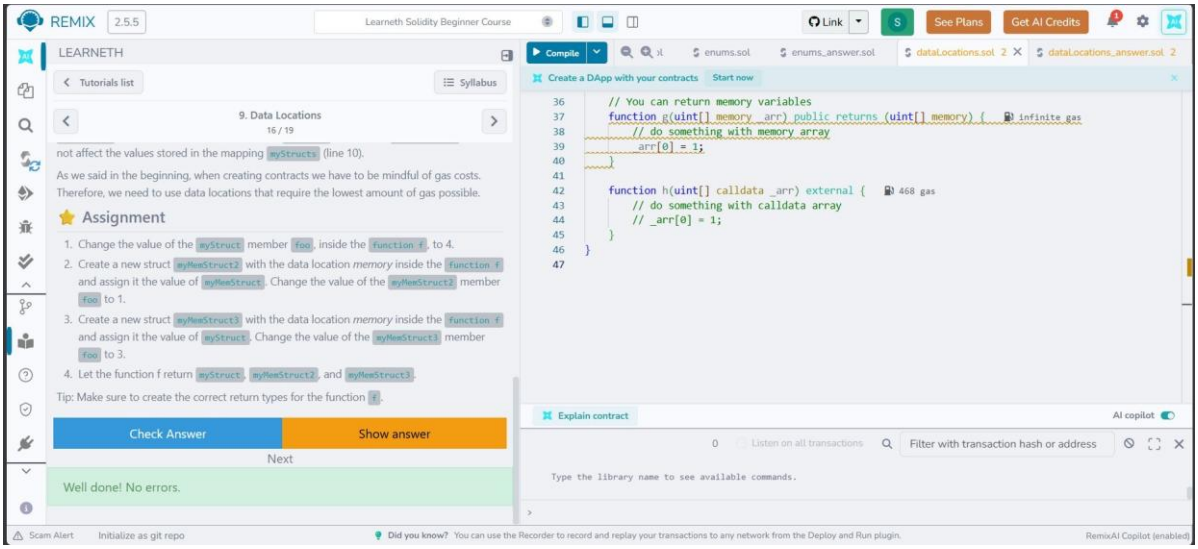


Figure 21 — Tutorial No. 18 execution in Remix IDE. **Tutorial**

No. 19

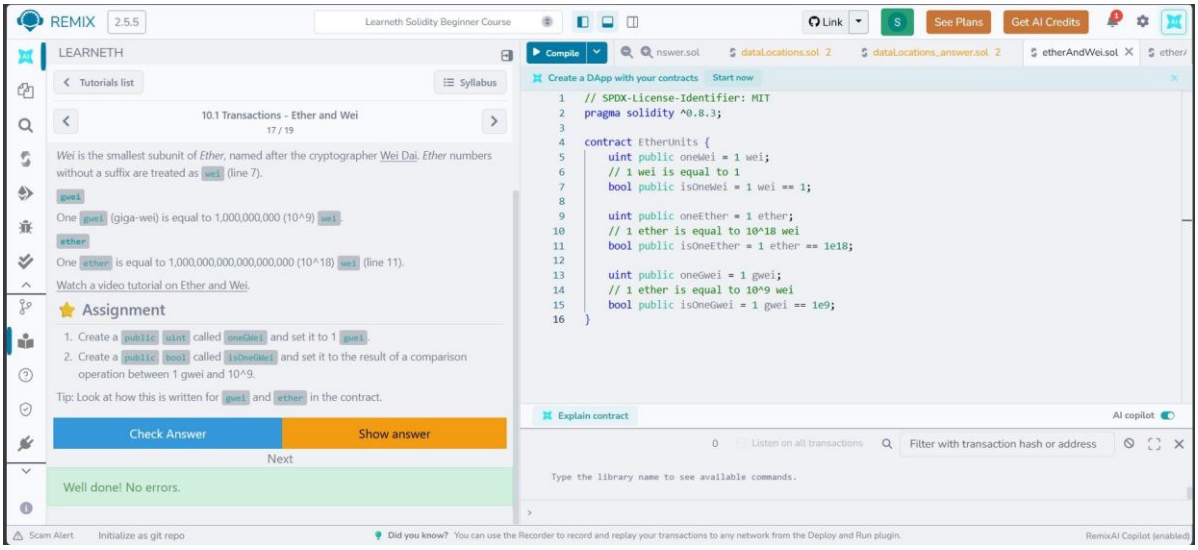


Figure 22 — Tutorial No. 19 execution in Remix IDE.

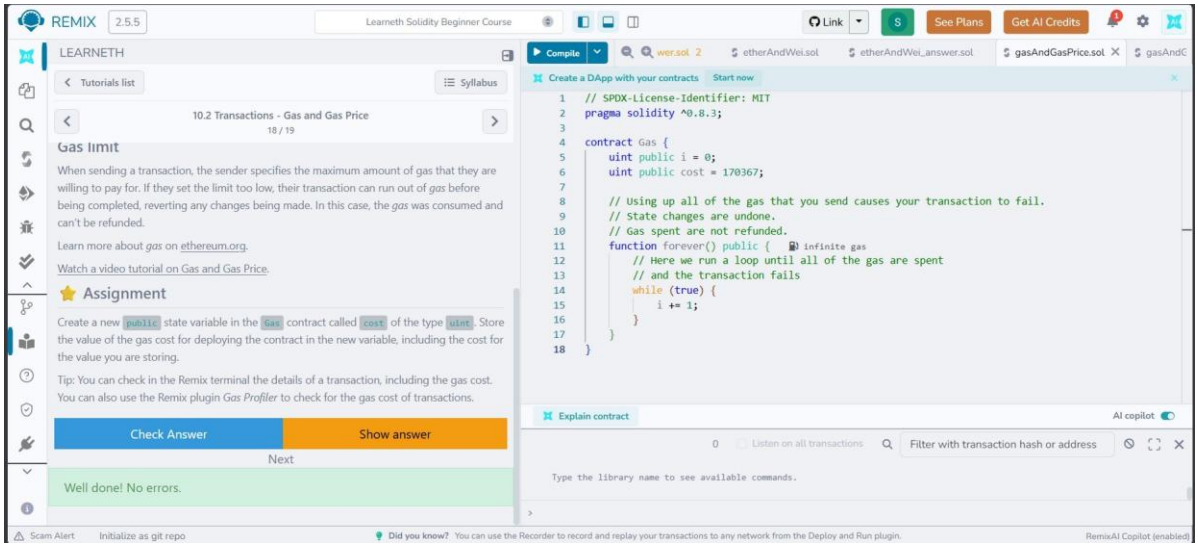


Figure 23 — Tutorial No. 19 execution in Remix IDE.

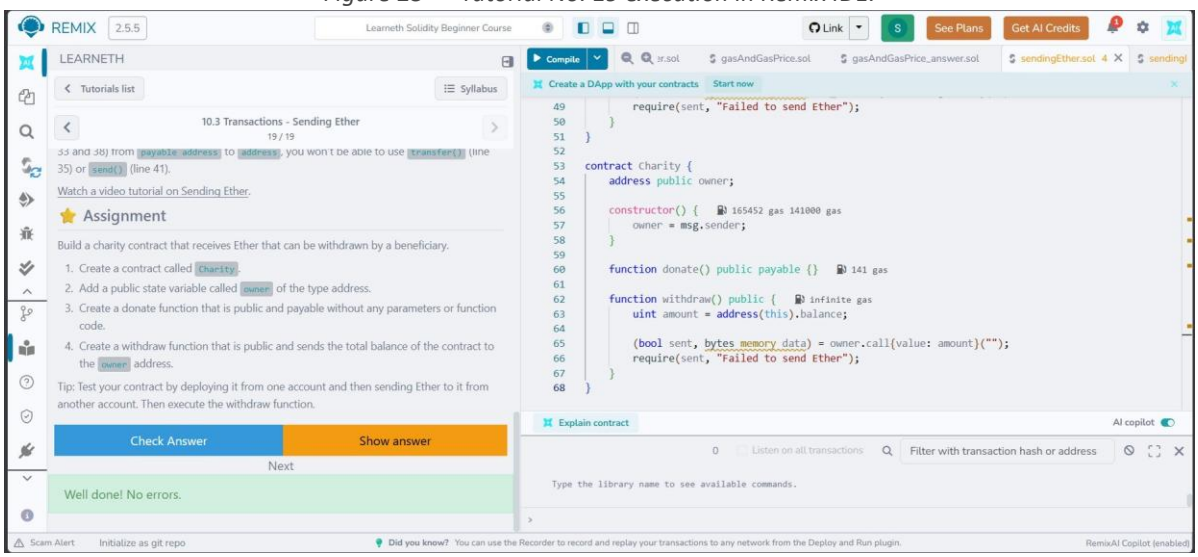


Figure 24 — Tutorial No. 19 execution in Remix IDE.

Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The handson practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.